Understand what you are building

User / Browser / curl

Sends HTTP requests to the application

NGINX Load Balancer

Port 8080 · Routes requests · Checks backend health

Simulated AZ-A

Web Server 1

Container: web-a

Response: Instance 1 – AZ-A

/health
Simulated AZ-B

Web Server 2

Container: web-b

Response: Instance 2 – AZ-B

/health

These are simulated availability zones, not real AWS zones. The two application servers will be separated into different Docker networks.

Important technical detail: Standard open-source NGINX does not provide native active HTTP health checks in its free version. We will use NGINX passive health checks, which detect certain failures while proxying requests. We can test stopped-server failover and recovery, but this is not identical to an AWS ALB actively polling `/health`.

For the assignment, document that limitation honestly. If your instructor explicitly requires periodic active health checks, we can add a separate health-check script and automate backend availability management.

Create your project folder

Open your Ubuntu terminal. If you use Windows with Docker Desktop, you can also run the commands in PowerShell, provided Docker Desktop is running.

```
mkdir -p ~/shopfast-ha/screenshots
cd ~/shopfast-ha
```

Your repository will eventually look like this:

```
shopfast-ha/
├── docker-compose.yml
├── nginx/
│   └── nginx.conf
├── web/
│   └── index.html
├── README.md
├── .gitignore
└── screenshots/
    ├── T1-health-status.png
    ├── T2-load-distribution.png
    ├── T3-browser-test.png
    ├── T4-az-failure.png
    ├── T5-recovery.png
    └── T6-access-logs.png
```

Build the two web servers

Create the web application file:

```
mkdir -p web nginx
nano web/index.html
```

Paste this HTML:

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>ShopFast High Availability</title>
  <meta charset="UTF-8">
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 100px;
      background: #f3f6fa;
    }
    .card {
      display: inline-block;
      background: white;
      padding: 40px;
      border-radius: 12px;
      box-shadow: 0 4px 12px #ccc;
    }
    h1 { color: #16794b; }
  </style>
</head>
<body>
  <div class="card">
    <h1>ShopFast</h1>
    <h2 id="instance">Instance</h2>
    <p id="zone">Availability Zone</p>
    <p>High-Availability Web Service</p>
  </div>

  <script>
    fetch('/instance-info')
      .then(response => response.json())
      .then(data => {
        document.getElementById('instance').textContent =
          data.instance;
        document.getElementById('zone').textContent =
          data.zone;
      });
  </script>
</body>
</html>
```

We will use NGINX itself to serve each instance's page and return its own identity through a dedicated endpoint. This keeps the example simple and avoids requiring a separate application runtime.

Create the following configuration for the web servers in `web/default.conf`:

Run nano web/default.conf

Then write the following…

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location = /health {
        access_log off;
        default_type text/plain;
        return 200 "healthy\n";
    }

    location = /instance-info {
        default_type application/json;
        return 200 '{"instance":"$hostname","zone":"$zone"}';
    }
}
```

Save the file

1. Press `Ctrl + O` to save.
2. Press `Enter` to confirm the filename.
3. Press `Ctrl + X` to exit.

We need the container's zone value to be configured through an environment variable. For that, we'll use a small template file instead. Replace the preceding `web/default.conf` with this version:

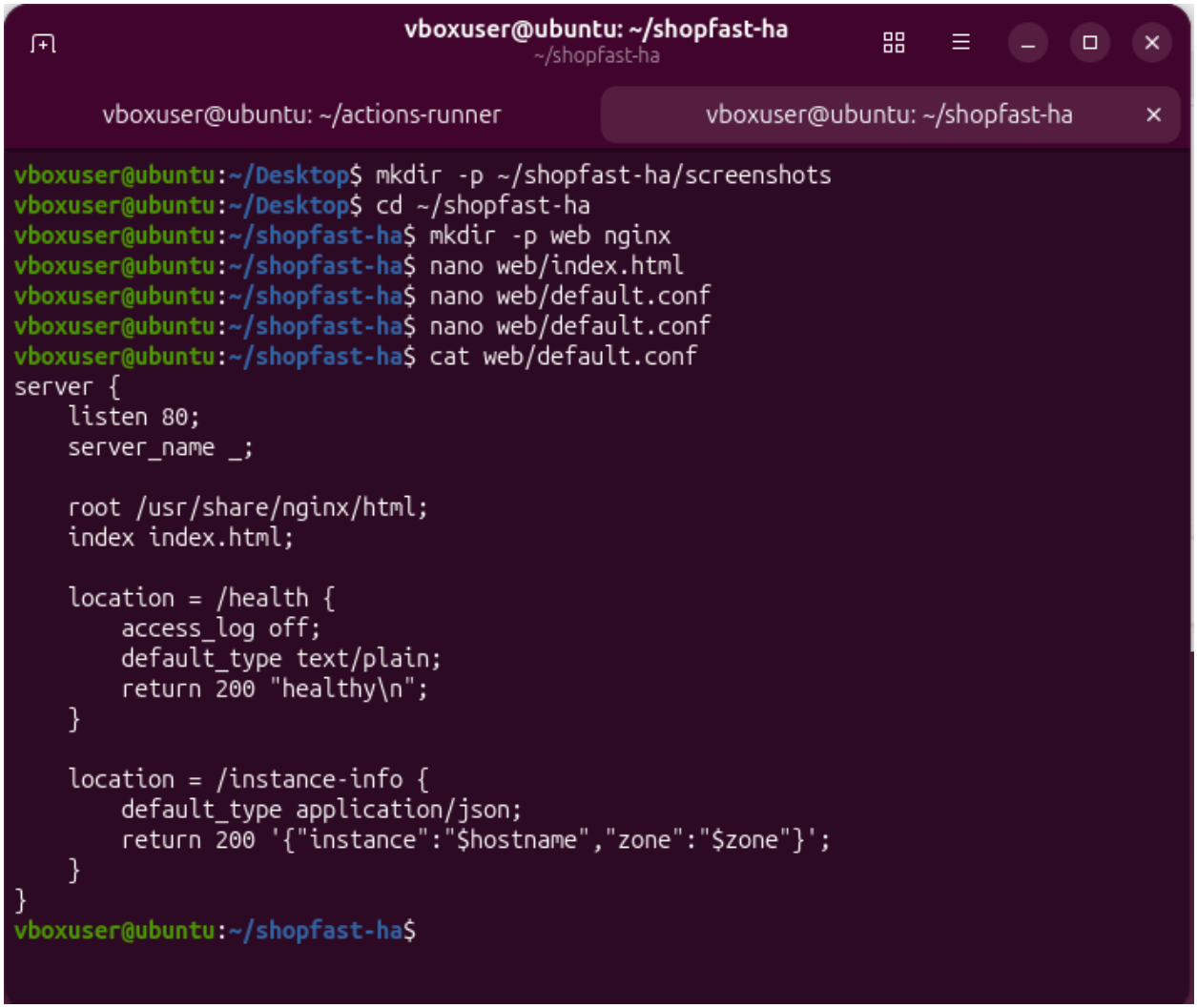
```
server {
    listen 80;
```

```
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location = /health {
        access_log off;
        default_type text/plain;
        return 200 "healthy\n";
    }

    location = /instance-info {
        default_type application/json;
        return 200 '{"instance":"${INSTANCE_NAME}","zone":"${ZONE_NAME}"}';
    }
}
```



Save it as `web/default.conf.template`. Docker's NGINX image can substitute the environment variables when the container starts.

Configure the NGINX load balancer

Create the load-balancer configuration:

nano nginx/nginx.conf

Paste:

```
events {}

http {
    log_format shopfast
        '$time_iso8601 client=$remote_addr '
        'backend=$upstream_addr status=$status '
        'upstream_status=$upstream_status '
        'request="$request"';

    access_log /var/log/nginx/access.log shopfast;

    upstream shopfast_backend {
        # Passive health checks in open-source NGINX
        server web-a:80 max_fails=1 fail_timeout=10s;
        server web-b:80 max_fails=1 fail_timeout=10s;
    }

    server {
        listen 80;
        server_name _;

        location / {
            proxy_pass http://shopfast_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;

            proxy_connect_timeout 2s;
            proxy_read_timeout 5s;
```

```
        proxy_next_upstream error timeout
            http_502 http_503 http_504;
        proxy_next_upstream_tries 2;
    }
  }
}
```

This configuration:

- Distributes requests between the two backend servers.
- Tracks failures using `max_fails` and `fail_timeout`.
- Attempts the other backend when a retryable upstream failure occurs.
- Logs which backend handled each request.

Create Docker Compose configuration

Create `docker-compose.yml` in your project root:

```
services:
  web-a:
    image: nginx:alpine
    container_name: shopfast-web-a
    environment:
      INSTANCE_NAME: "Instance 1"
      ZONE_NAME: "AZ-A"
    volumes:
      - ./web/index.html:/usr/share/nginx/html/index.html:ro
      - ./web/default.conf.template:/etc/nginx/templates/default.conf.template:ro
    networks:
      - az_a
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1/health"]
      interval: 5s
      timeout: 2s
      retries: 2
      start_period: 3s
    restart: unless-stopped
```

```
  web-b:
    image: nginx:alpine
    container_name: shopfast-web-b
    environment:
      INSTANCE_NAME: "Instance 2"
      ZONE_NAME: "AZ-B"
    volumes:
      - ./web/index.html:/usr/share/nginx/html/index.html:ro
      - ./web/default.conf.template:/etc/nginx/templates/default.conf.template:ro
    networks:
      - az_b
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1/health"]
      interval: 5s
      timeout: 2s
      retries: 2
      start_period: 3s
    restart: unless-stopped

  load-balancer:
    image: nginx:alpine
    container_name: shopfast-lb
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - az_a
      - az_b
    depends_on:
      web-a:
        condition: service_healthy
      web-b:
        condition: service_healthy
    restart: unless-stopped

networks:
  az_a:
    name: shopfast-az-a
    driver: bridge
  az_b:
    name: shopfast-az-b
    driver: bridge
```

The web servers have no published host ports. Only the load balancer exposes port `8080`. Each web server is on a separate Docker network, and the load balancer is connected to both networks so it can reach both servers.

The Docker health checks report backend health, but standard NGINX does not automatically consume Docker's health status. Its upstream failure handling is passive. This distinction is important when you document your implementation.

Install Docker on Ubuntu 🐳

I'll show you how to install Docker on Ubuntu step by step. We'll install **Docker Engine and the Docker Compose plugin** so you can run your `docker-compose.yml` file.

Update Ubuntu

Open your terminal using `Ctrl + Alt + T` and run:

sudo apt update

Wait for the update to finish.

Install required packages

Run:

sudo apt install ca-certificates curl -y

These packages are needed to securely download Docker's official repository key.

Add Docker's official GPG key

Run:

sudo install -m 0755 -d /etc/apt/keyrings

Then download the Docker signing key:

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc

Set the correct permissions:

sudo chmod a+r /etc/apt/keyrings/docker.asc

Add the Docker repository

Copy and run this entire command:

echo \

"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \

$(. /etc/os-release && echo "{UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | \

sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

This tells Ubuntu where to download Docker packages.

Update the package list

Run:

sudo apt update

Install Docker

Run this command to install Docker Engine, the Docker CLI, and Docker Compose:

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y

This installs Docker and the Compose plugin from Docker's official repository.【turn0search0】

Start Docker

Run:

sudo systemctl start docker

Enable Docker to start automatically when Ubuntu boots:

sudo systemctl enable docker

Check that Docker is running

Run:

sudo systemctl status docker

If it says `active (running)`, continue to Step 2.

If Docker isn't running, start it:

sudo systemctl start docker

Add `vboxuser` to the Docker group

Run this command:

sudo usermod -aG docker vboxuser

Apply the changes

Run:

newgrp docker

Verify the installation

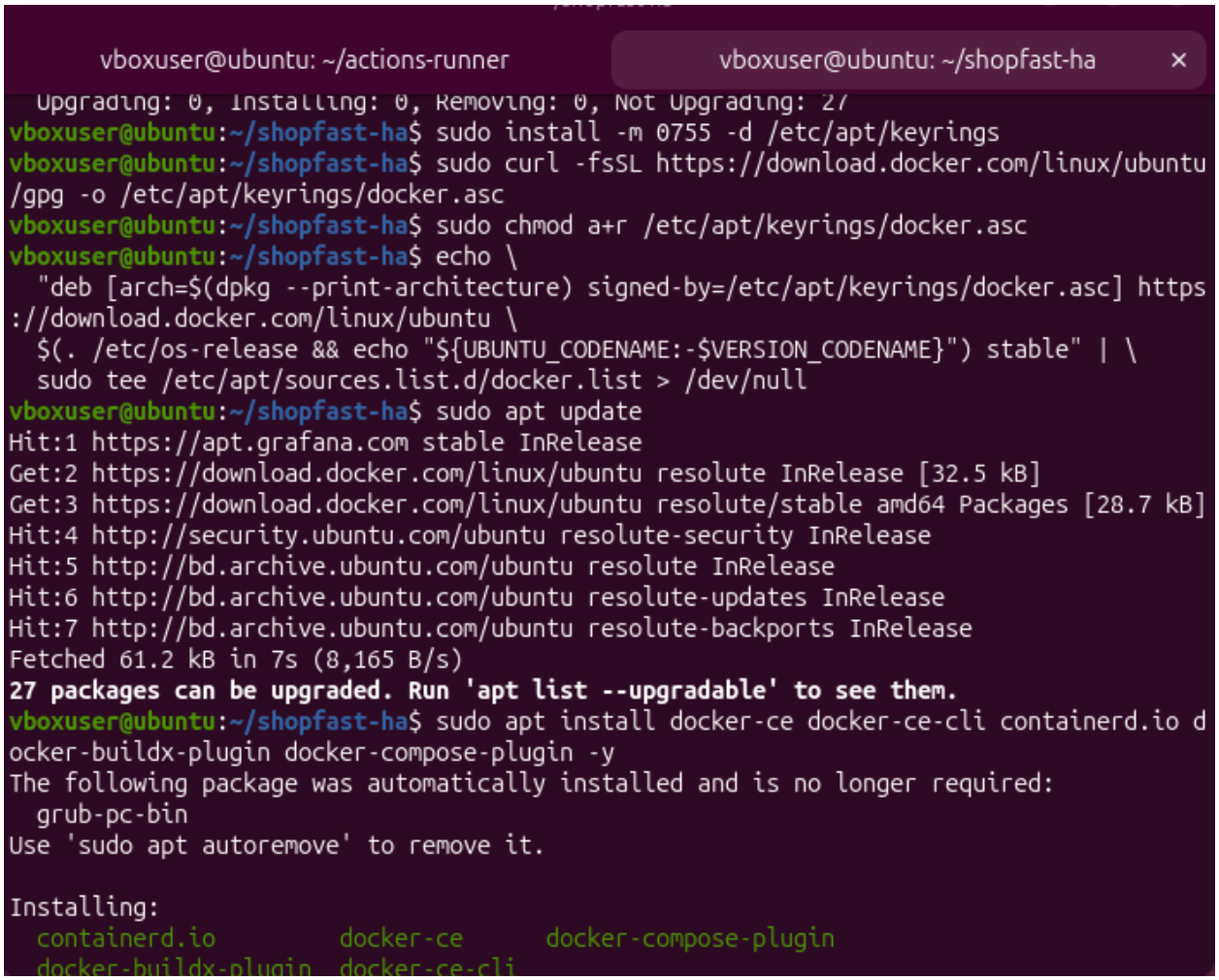
Check the Docker version:

docker --version

```
vboxuser@ubuntu:~/shopfast-ha$ pwd
/home/vboxuser/shopfast-ha
vboxuser@ubuntu:~/shopfast-ha$ ls
nginx  screenshots  web
vboxuser@ubuntu:~/shopfast-ha$ touch docker-compose.yml
vboxuser@ubuntu:~/shopfast-ha$ nano docker-compose.yml
vboxuser@ubuntu:~/shopfast-ha$ cat docker-compose.yml
services:
  web-a:
    image: nginx:alpine
    container_name: shopfast-web-a
    environment:
      INSTANCE_NAME: "Instance 1"
      ZONE_NAME: "AZ-A"
    volumes:
      - ./web/index.html:/usr/share/nginx/html/index.html:ro
      - ./web/default.conf.template:/etc/nginx/templates/default.conf.template:ro
    networks:
      - az_a
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1/health"]
      interval: 5s
      timeout: 2s
      retries: 2
      start_period: 3s
    restart: unless-stopped

  web-b:
    image: nginx:alpine
```

Start the application



From the `shopfast-ha` directory, run:

Check docker is running:

sudo systemctl status docker

```
ChatGPT: command not found
vboxuser@ubuntu:~/shopfast-ha$ sudo systemctl start docker
vboxuser@ubuntu:~/shopfast-ha$ sudo usermod -aG docker vboxuser
vboxuser@ubuntu:~/shopfast-ha$ newgrp docker
Command 'newgrp' not found, but can be installed with:
sudo apt install util-linux-extra
vboxuser@ubuntu:~/shopfast-ha$ sudo apt install util-linux-extra
The following package was automatically installed and is no longer required:
  grub-pc-bin
Use 'sudo apt autoremove' to remove it.

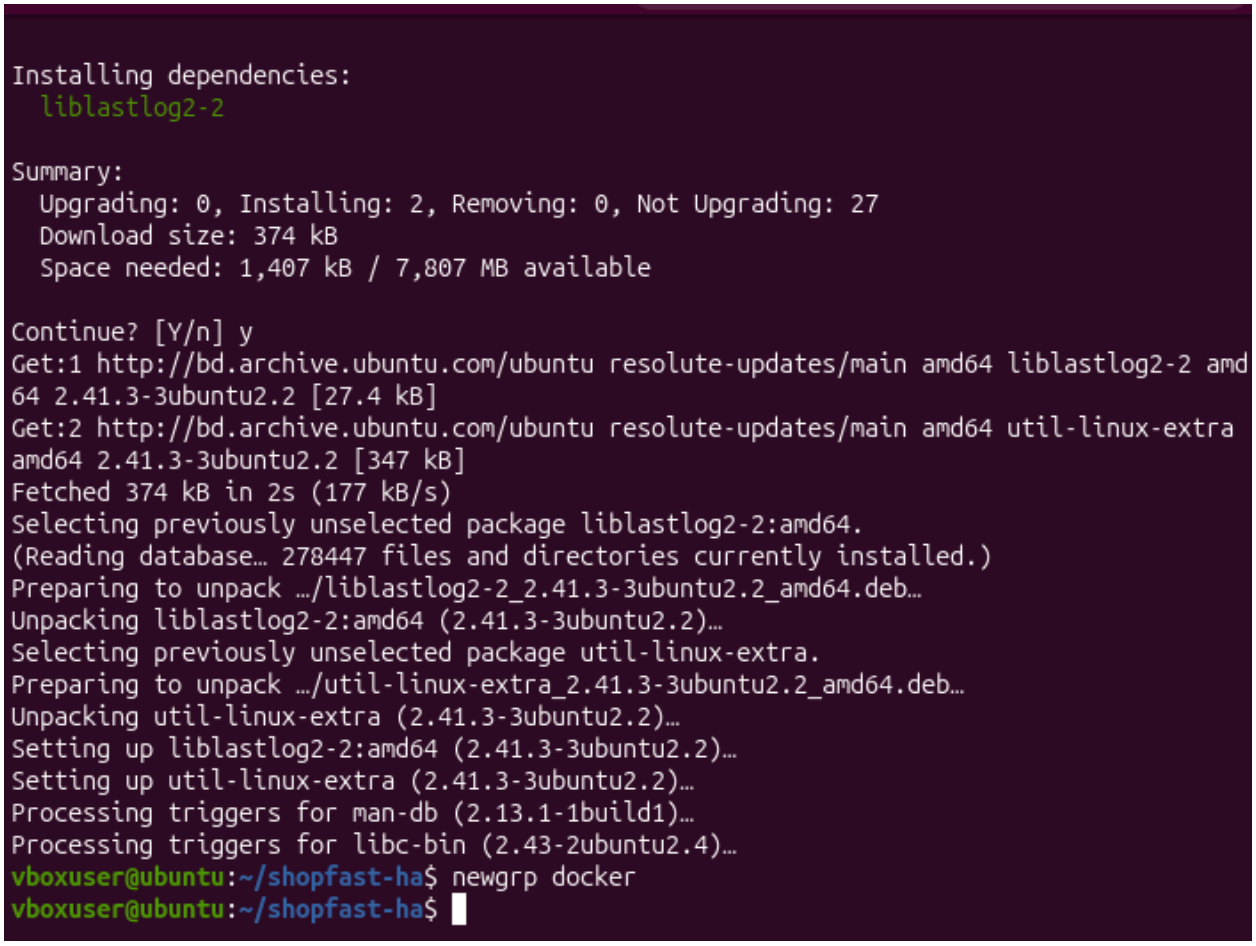
Installing:
  util-linux-extra

Installing dependencies:
  liblastlog2-2

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 27
  Download size: 374 kB
```

```
vboxuser@ubuntu:~/shopfast-ha$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-10-11 03:25:51 UTC; 11min ago
 Invocation: 710707bf33684070b906f8c662794801
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 36262 (dockerd)
      Tasks: 12
     Memory: 111M (peak: 119.7M)
        CPU: 7.728s
     CGroup: /system.slice/docker.service
             └─36262 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd>

Oct 11 03:25:51 ubuntu dockerd[36262]: time="2026-10-11T03:25:51.196330917Z" level=inf>
Oct 11 03:25:51 ubuntu dockerd[36262]: time="2026-10-11T03:25:51.196481729Z" level=inf>
Oct 11 03:25:51 ubuntu systemd[1]: Started docker.service - Docker Application Contain>
Oct 11 03:27:12 ubuntu dockerd[36262]: time="2026-10-11T03:27:12.344869510Z" level=inf>
Oct 11 03:27:13 ubuntu dockerd[36262]: time="2026-10-11T03:27:13.268301168Z" level=inf>
Oct 11 03:31:27 ubuntu dockerd[36262]: time="2026-10-11T03:31:27.492056443Z" level=inf>
Oct 11 03:31:27 ubuntu dockerd[36262]: time="2026-10-11T03:31:27.508674500Z" level=inf>
Oct 11 03:31:27 ubuntu dockerd[36262]: time="2026-10-11T03:31:27.814055672Z" level=inf>
Oct 11 03:31:30 ubuntu dockerd[36262]: time="2026-10-11T03:31:30.120607278Z" level=inf>
Oct 11 03:31:30 ubuntu dockerd[36262]: time="2026-10-11T03:31:30.718222711Z" level=inf>
lines 1-23/23 (END)
```



Step 2: Check the template

Run:

cat web/default.conf.template

This is the important step. Your health check requests:

http://127.0.0.1/health

Your `web/default.conf` already defines `/health` correctly, returning HTTP 200. However, the running containers are returning HTTP 404, which suggests Nginx may be using a different configuration.

Step 3: Fix the template if necessary

If `web/default.conf.template` does not exist, create it:

nano web/default.conf.template

Paste the following configuration:

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location = /health {
        access_log off;
        default_type text/plain;
        return 200 "healthy\n";
    }

    location = /instance-info {
        default_type application/json;
```

```
        return 200 '{"instance":"$INSTANCE_NAME", "zone": "{ZONE_NAME}"}';
    }

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Save with `Ctrl + O`, press `Enter`, then exit with `Ctrl + X`.

This configuration includes:

- `/health` — returns HTTP 200 for Docker health checks.
- `/instance-info` — returns instance and availability-zone information.
- `/` — serves your `index.html` file.

Restart the containers

After confirming the template is correct, run:

```
docker compose up -d --force-recreate
```

Check the status:

```
docker compose ps
```

Both `shopfast-web-a` and `shopfast-web-b` should eventually show `Up (healthy)`.

Test the health endpoint

Run:

```
docker exec shopfast-web-a wget -q -O- http://127.0.0.1/health
```

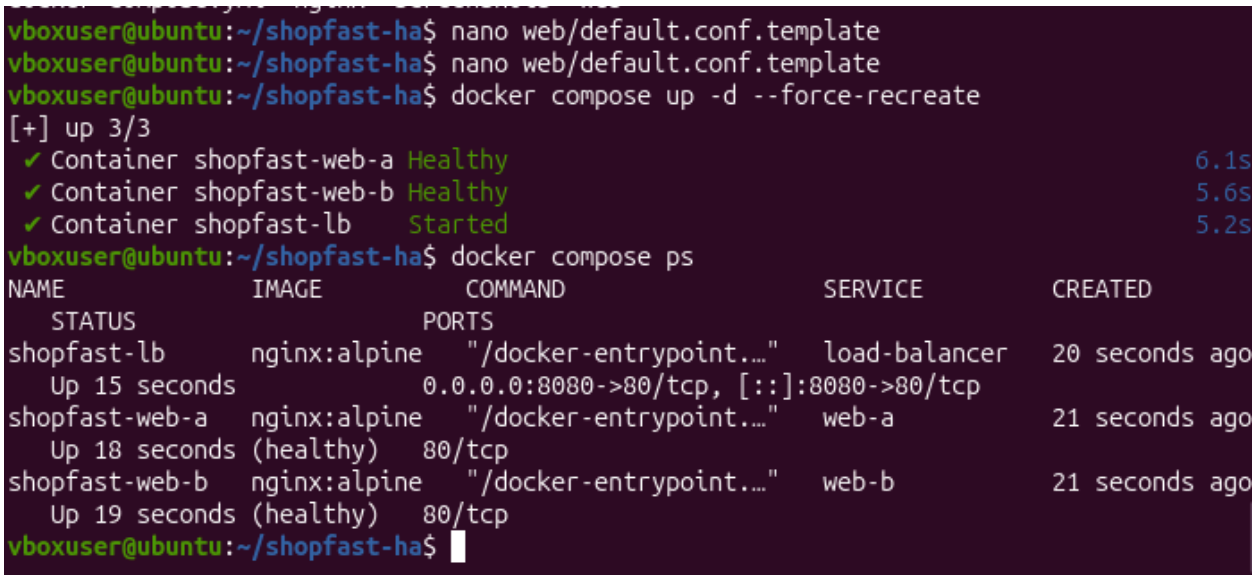
Expected output:

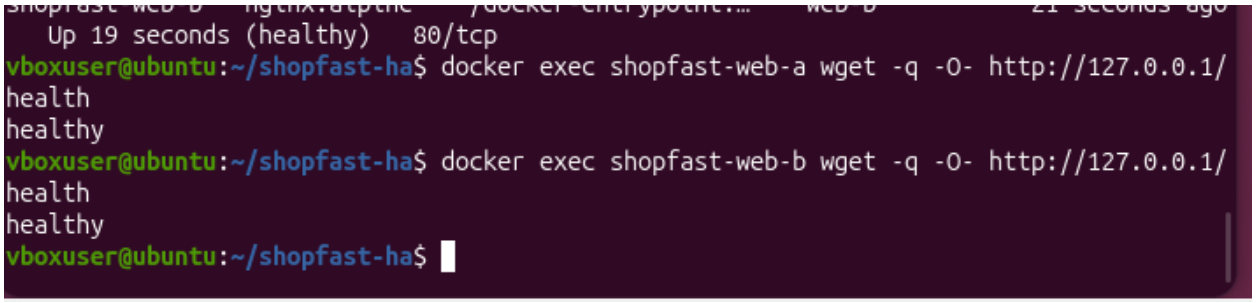
healthy

Repeat for the second container:

docker exec shopfast-web-b wget -q -O- http://127.0.0.1/health

One important point: Before making changes, run `ls -l web/` and `cat web/default.conf.template`. If the template already exists, let's inspect it first rather than overwrite your existing configuration.





docker compose config
docker compose up -d
docker compose ps

```
vboxuser@ubuntu:~/shopfast-ha$ docker compose up -d
[+] up 3/3
 ✔ Container shopfast-lb     Running                                          0.
 ✔ Container shopfast-web-a  Healthy                                          0.
 ✔ Container shopfast-web-b  Healthy                                          0.
vboxuser@ubuntu:~/shopfast-ha$ docker compose ps
NAME             IMAGE          COMMAND                  SERVICE         CREATED
 STATUS                  PORTS
shopfast-lb      nginx:alpine   "/docker-entrypoint.…"   load-balancer   11 hours ago
 Up 11 hours             0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
shopfast-web-a   nginx:alpine   "/docker-entrypoint.…"   web-a           11 hours ago
 Up 11 hours (healthy)   80/tcp
shopfast-web-b   nginx:alpine   "/docker-entrypoint.…"   web-b           11 hours ago
 Up 11 hours (healthy)   80/tcp
vboxuser@ubuntu:~/shopfast-ha$
```

Wait until both web containers are healthy and the load balancer is running.

Validate the NGINX configuration:

docker exec shopfast-lb nginx -t

Check each backend's health endpoint from its own container:

docker exec shopfast-web-a wget -qO- http://127.0.0.1/health
docker exec shopfast-web-b wget -qO- http://127.0.0.1/health

```
vboxuser@ubuntu:~/shopfast-ha$ docker exec shopfast-lb nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
vboxuser@ubuntu:~/shopfast-ha$ docker exec shopfast-web-a wget -qO- http://127.0.0.1/he
alth
healthy
vboxuser@ubuntu:~/shopfast-ha$ docker exec shopfast-web-b wget -qO- http://127.0.0.1/he
alth
healthy
vboxuser@ubuntu:~/shopfast-ha$
```

Both should return:

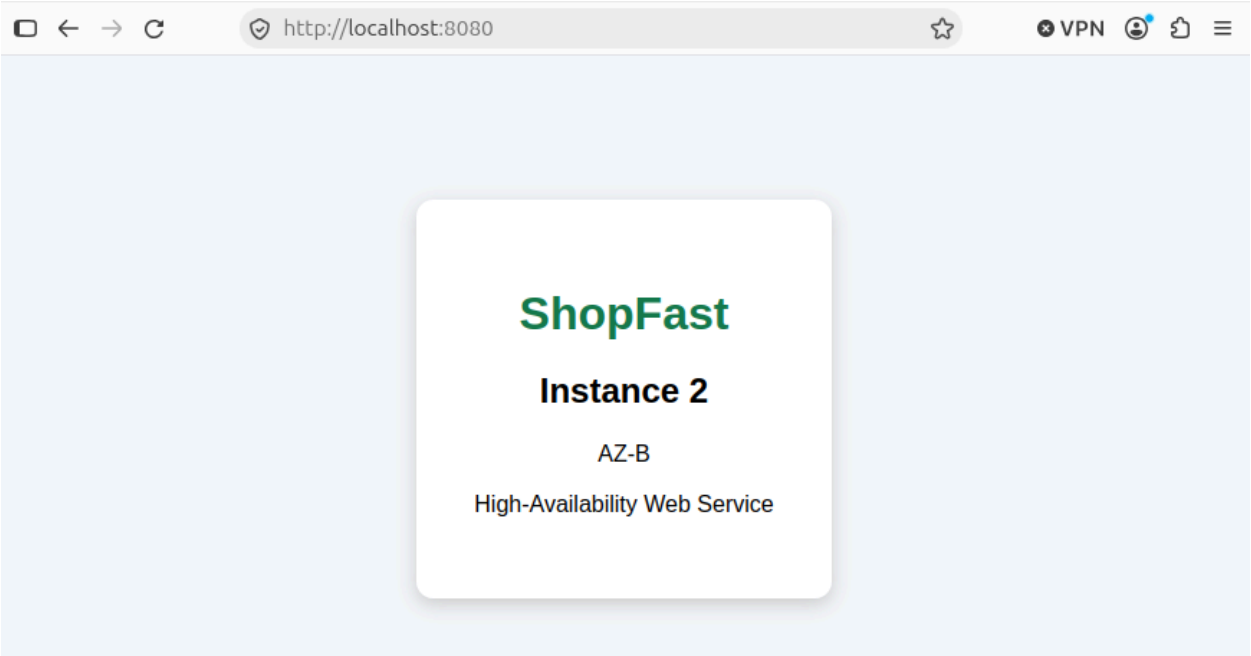
healthy

Open this address in your browser:

Local application URL

`http://localhost:8080`

You should see the ShopFast page. The instance information is loaded from `/instance-info` and should identify the backend that served the page.



Test load distribution — T2

Run 10 requests through the load balancer:

```
for i in $(seq 1 10); do
  curl -s http://localhost:8080/instance-info
  echo
done
```

Expected output will resemble:

```
{"instance":"Instance 1","zone":"AZ-A"}
{"instance":"Instance 2","zone":"AZ-B"}
```

{"instance":"Instance 1","zone":"AZ-A"}
...

With the default round-robin strategy and both servers healthy, requests should be distributed across both servers. The exact order can vary because of retries or failures.



Check both instances — T1

Run:

docker compose ps

For more detail:

```
docker inspect --format='{{.Name}}: {{.State.Health.Status}}' \
  shopfast-web-a shopfast-web-b
```

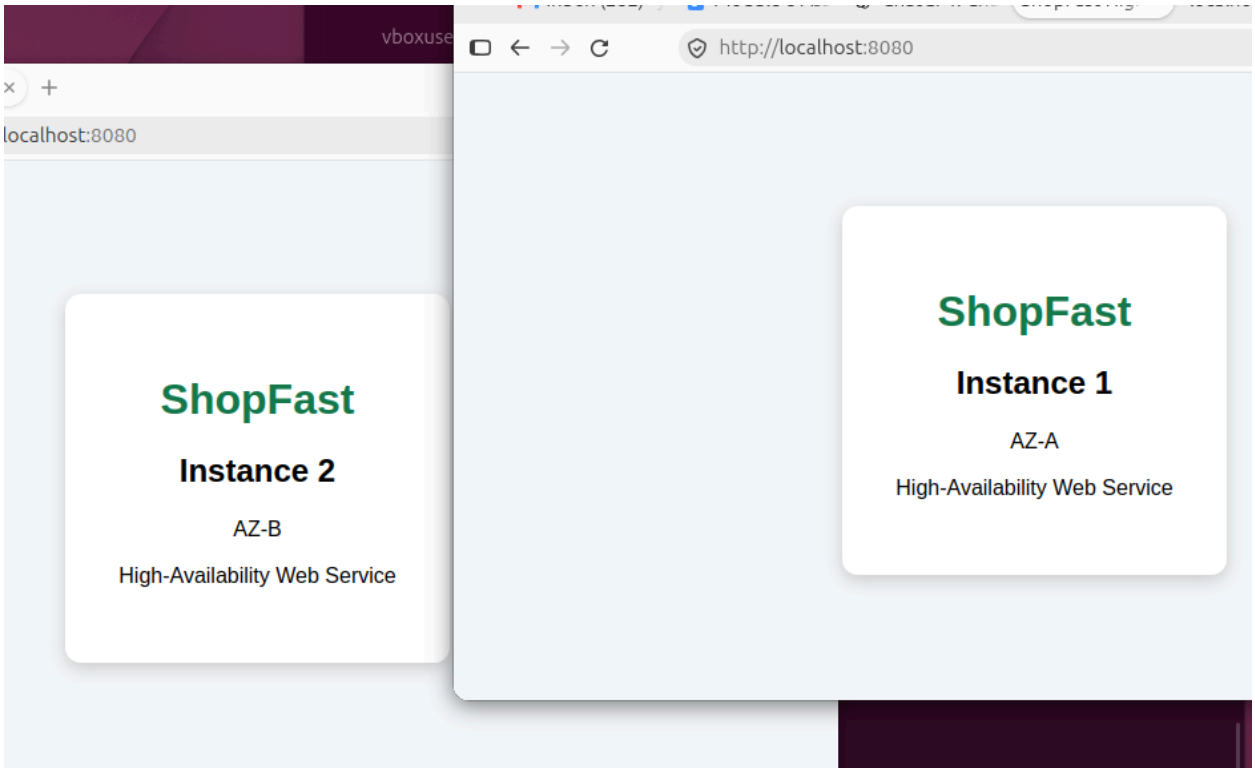
Expected:

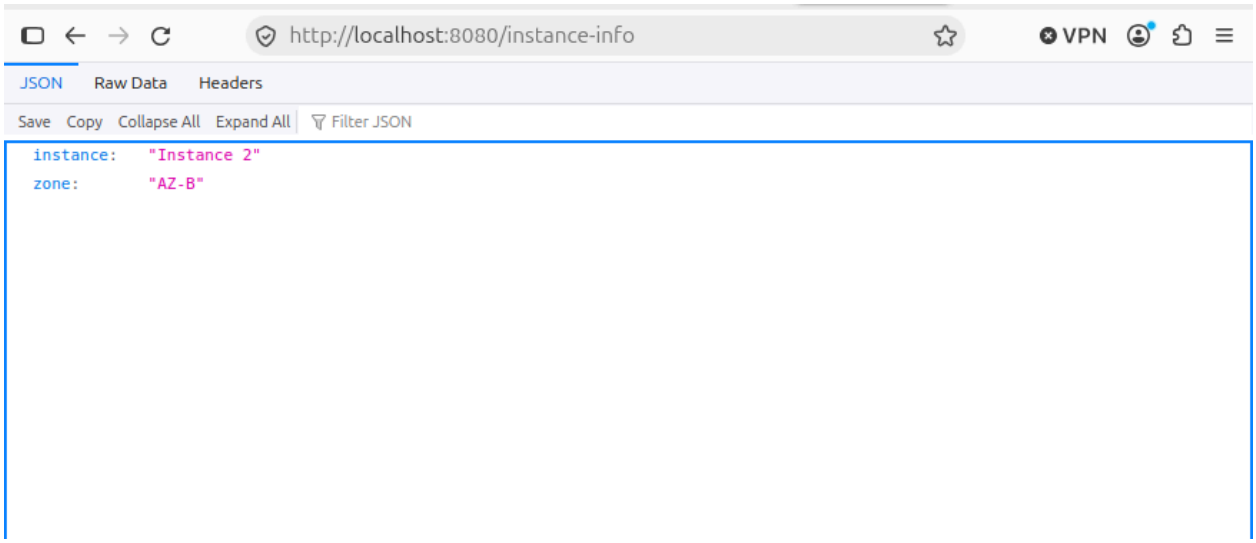
/shopfast-web-a: healthy
/shopfast-web-b: healthy

```
vboxuser@ubuntu:~/shopfast-ha$ docker compose ps
NAME              IMAGE          COMMAND                  SERVICE         CREATED
 STATUS                  PORTS
shopfast-lb       nginx:alpine   "/docker-entrypoint.…"   load-balancer   12 hours ago
 Up 12 hours             0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
shopfast-web-a    nginx:alpine   "/docker-entrypoint.…"   web-a           12 hours ago
 Up 12 hours (healthy)   80/tcp
shopfast-web-b    nginx:alpine   "/docker-entrypoint.…"   web-b           12 hours ago
 Up 12 hours (healthy)   80/tcp
vboxuser@ubuntu:~/shopfast-ha$ docker inspect --format='{{.Name}}: {{.State.Health.Stat
us}}' \
  shopfast-web-a shopfast-web-b
/shopfast-web-a: healthy
/shopfast-web-b: healthy
vboxuser@ubuntu:~/shopfast-ha$
```

Test the browser — T3

1. Open `http://localhost:8080`.
2. Refresh several times.
3. Observe the instance and AZ shown on the page.
4. You can also visit `http://localhost:8080/instance-info` to see the backend identity directly.





Simulate an Availability Zone failure — T4

Stop the first web server:

docker compose stop web-a

Check its status:

docker compose ps -a

Send repeated requests:

```
for i in $(seq 1 10); do
  curl --max-time 5 -sS http://localhost:8080/instance-info
  echo
done
```

```
vboxuser@ubuntu:~/shopfast-ha$ docker compose stop web-a
[+] stop 1/1
 ✔ Container shopfast-web-a Stopped                                          0.8s
vboxuser@ubuntu:~/shopfast-ha$ docker compose ps -a
NAME             IMAGE          COMMAND                  SERVICE         CREATED
 STATUS                        PORTS
shopfast-lb      nginx:alpine   "/docker-entrypoint.…"   load-balancer   12 hours ago
 Up 12 hours                   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
shopfast-web-a   nginx:alpine   "/docker-entrypoint.…"   web-a           12 hours ago
 Exited (0) 15 seconds ago
shopfast-web-b   nginx:alpine   "/docker-entrypoint.…"   web-b           12 hours ago
 Up 12 hours (healthy)         80/tcp
vboxuser@ubuntu:~/shopfast-ha$ for i in $(seq 1 10); do
  curl --max-time 5 -sS http://localhost:8080/instance-info
  echo
done
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 2","zone":"AZ-B"}
vboxuser@ubuntu:~/shopfast-ha$
```

The expected steady-state behavior is that requests are served by AZ-B while AZ-A is unavailable.

Test recovery — T5

Start the failed server again:

docker compose start web-a

Wait for the health check to recover:

```
until [ "$(docker inspect -f '{{.State.Health.Status}}' shopfast-web-a)" = "healthy" ]; do
  sleep 2
done
```

Now send requests again:

for i in $(seq 1 10); do

```
  curl -s http://localhost:8080/instance-info
  echo
done
```

```
vboxuser@ubuntu:~/shopfast-ha$ docker compose start web-a
[+] start 1/1
 ✔ Container shopfast-web-a Started                                    1.1s
vboxuser@ubuntu:~/shopfast-ha$ until [ "$(docker inspect -f '{{.State.Health.Status}}'
shopfast-web-a)" = "healthy" ]; do
  sleep 2
done
vboxuser@ubuntu:~/shopfast-ha$ for i in $(seq 1 10); do
  curl -s http://localhost:8080/instance-info
  echo
done
{"instance":"Instance 1","zone":"AZ-A"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 1","zone":"AZ-A"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 1","zone":"AZ-A"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 1","zone":"AZ-A"}
{"instance":"Instance 2","zone":"AZ-B"}
{"instance":"Instance 1","zone":"AZ-A"}
{"instance":"Instance 2","zone":"AZ-B"}
vboxuser@ubuntu:~/shopfast-ha$
```

After the backend is reachable again, NGINX should resume sending traffic to it.

Check logs — T6

View NGINX access logs:

docker compose logs --tail=50 load-balancer

Or follow the logs live:

docker compose logs -f load-balancer

The access log includes the selected upstream address, HTTP status and upstream status, allowing you to see which backend handled each request.

Save a screenshot as `screenshots/T6-access-logs.png`.

Complete Problem 1 — Architecture and component mapping

Add this table to your README.

AWS component	Local equivalent
Application Load Balancer	NGINX container
Availability Zone A	Docker bridge network `shopfast-az-a`
Availability Zone B	Docker bridge network `shopfast-az-b`
EC2 Instance 1	`shopfast-web-a` container
EC2 Instance 2	`shopfast-web-b` container
Target Group	NGINX upstream `shopfast_backend`
Health Check	Docker HTTP health check plus NGINX passive failure detection
Listener	NGINX listening on container port 80, published as host port 8080

Explain that these are simulated AZs. Separate Docker networks isolate backend connectivity, but both containers still run on the same host, so this does not protect against host failure.

Complete Problem 4 — Health-check explanation

Use these values in your report:

Setting	Value	Reason
Health endpoint	`/health`	Simple endpoint to verify the web server responds
Docker check interval	5 seconds	Detects local container health changes promptly
Docker timeout	2 seconds	Avoids waiting too long for a response
Docker retries	2	Allows a brief transient failure
NGINX `max_fails`	1	One qualifying upstream failure marks the server unavailable temporarily
NGINX `fail_timeout`	10 seconds	Failure tracking and temporary exclusion period
Proxy timeout	2 seconds to connect	Limits connection waiting time

Active vs. passive health checks

- Active health checks: The load balancer periodically sends requests to an endpoint such as `/health`, even when no user requests arrive.

- Passive health checks: The load balancer observes failures during actual user requests and temporarily avoids a failing server.

In this implementation, Docker performs active local container health checks, while open-source NGINX uses passive upstream failure detection. These mechanisms are separate; Docker's unhealthy status does not automatically remove the container from NGINX's upstream pool.

Publish the project on GitHub

Create a public repository named `shopfast-ha`, then run these commands in your project directory:

```
git init
git branch -M main
git add .
git commit -m "Initial ShopFast project structure"
git remote add origin https://github.com/jubayerdevs/shopfast-ha.git
git push -u origin main
```